

Ejemplos teórico-prácticos de apoyo

Contents

- Los datos
- ¿Qué pasa si intentamos un *merge* exacto?
- PASO 1: Calcular la mejor coincidencia para cada nombre de `df_beneficiarios`
- PASO 2: Merge usando la columna de coincidencias

! Sobre esta sección

En esta sección pueden encontrar una *notebook* con código para realizar *fuzzy joins*, basado en el ejemplo que se presenta en los apuntes y en las diapositivas de clase.

Supongamos que trabajamos en la Secretaría de Desarrollo Social de un municipio y necesitamos cruzar dos bases de datos: el padrón electoral, que contiene información de los vecinos registrados, y el registro de beneficiarios de un programa de subsidios.

Ambas bases fueron cargadas de manera independiente y en momentos distintos, por lo que los nombres no siempre coinciden exactamente: pueden faltar tildes, aparecer abreviaturas, existir errores tipográficos o incluso haber personas que figuran en una base pero no en la otra.

Nuestro objetivo es vincular a los beneficiarios del programa con los registros del padrón. Para eso, y como veremos enseguida, un *merge* exacto no alcanza.

```
# Librerías
import pandas as pd
from rapidfuzz import process, fuzz, utils
```

Los datos

`df_padron`: contiene el registro oficial de vecinos del municipio.

```
df_padron = pd.DataFrame({'Nombre': ['María González', 'Carlos Alberto Rodríguez', 'Luisa  
'DNI': [28456123, 31782456, 25963147, 33841290, 29571834],  
'Localidad': ['Rosario', 'Rosario', 'Villa Gobernador Gálvez', 'Rosario', 'Funes']})  
  
print(df_padron)
```

	Nombre	DNI	Localidad
0	María González	28456123	Rosario
1	Carlos Alberto Rodríguez	31782456	Rosario
2	Luisa Fernanda Martínez	25963147	Villa Gobernador Gálvez
3	Roberto Sánchez	33841290	Rosario
4	Ana Laura Pérez	29571834	Funes

`df_beneficiarios`: contiene el registro de personas que reciben un subsidio municipal. Fue cargado a partir de formularios en papel digitalizados, por lo que presenta inconsistencias en los nombres: tildes faltantes, abreviaciones y errores tipográficos.

```
df_beneficiarios = pd.DataFrame({'Nombre': ['Maria Gonzalez', 'Carlos Rodríguez', 'L. F. M  
'Subsidio': [15000, 22000, 18000, 12000, 9000],  
'Fecha_Alta': ['2023-03-01', '2023-05-14', '2022-11-30', '2024-01-08', '2023-07-22']})  
  
print(df_beneficiarios)
```

	Nombre	Subsidio	Fecha_Alta
0	Maria Gonzalez	15000	2023-03-01
1	Carlos Rodríguez	22000	2023-05-14
2	L. F. Martínez	18000	2022-11-30
3	Roberto Sanches	12000	2024-01-08
4	Jorge Medina	9000	2023-07-22

¿Qué pasa si intentamos un *merge* exacto?

```
print(df_padron.merge(df_beneficiarios, on='Nombre'))
```

```
Empty DataFrame  
Columns: [Nombre, DNI, Localidad, Subsidio, Fecha_Alta]  
Index: []
```

El resultado es un DataFrame vacío: ningún nombre coincide de forma exacta entre ambas bases. Esto muestra por qué, cuando los datos provienen de fuentes distintas o fueron

cargados manualmente, un *merge* exacto no siempre es suficiente.

PASO 1: Calcular la mejor coincidencia para cada nombre de `df_beneficiarios`

En lugar de exigir coincidencias exactas, vamos a identificar, para cada nombre de `df_beneficiarios`, cuál es el nombre más similar en `df_padron` y con qué *score* de similitud.

Para eso definimos una función que utiliza `process.extractOne()` de `rapidfuzz`. Esta función recibe un nombre de búsqueda (`query`), una lista de candidatos (`choices`) y un scorer, y devuelve el candidato más similar junto con su *score*. Si ningún candidato supera el umbral mínimo indicado, devuelve `None`.

```
def buscar_coincidencia(nombre, opciones, umbral = 80):
    resultado = process.extractOne(
        nombre,
        opciones,
        scorer = fuzz.token_sort_ratio,
        processor = utils.default_process,
        score_cutoff = umbral
    )
    if resultado:
        return resultado[0], round(resultado[1],2)
    return "Ningún match", "No aplica"
```

Algunos parámetros clave que podemos modificar:

- `scorer`: define cómo se compara una cadena con otra. Aquí usamos `fuzz.token_sort_ratio`, que tokeniza las cadenas, ordena los *tokens* y luego calcula la similitud.
- `processor`: con `utils.default_process` se aplican tareas de limpieza previas antes de comparar.
- `score_cutoff`: fija el umbral mínimo para considerar que hubo coincidencia.

Aplicamos la función a cada fila de `df_beneficiarios`:

```
df_beneficiarios[['Nombre_matched', 'Score']] = df_beneficiarios['Nombre'].apply(
    lambda x: pd.Series(buscar_coincidencia(x, df_padron['Nombre']))
)

print(df_beneficiarios[['Nombre', 'Nombre_matched', 'Score']])
```

	Nombre	Nombre_matched	Score
0	Maria Gonzalez	María González	85.71
1	Carlos Rodríguez	Carlos Alberto Rodríguez	80.0
2	L. F. Martínez	Ningún match	No aplica
3	Roberto Sanches	Roberto Sánchez	86.67
4	Jorge Medina	Ningún match	No aplica

La tabla resultante muestra, para cada beneficiario, cuál es el nombre del padrón con el que mejor coincide y cuál es su nivel de similitud. Los casos cuyo `score` no supera el umbral quedan excluidos: en `Nombre_matched` aparece "Ningún match" y en `Score`, "No aplica".

Para pensar...



PASO 2: Merge usando la columna de coincidencias

Una vez creada la columna `Nombre_matched`, podemos usarla como clave para hacer el merge con `df_padrón`. Los registros sin coincidencia quedarán excluidos del resultado final.

```
df_final = df_beneficiarios.merge(df_padrón, left_on='Nombre_matched', right_on='Nombre').
print(df_final)
```

	Subsidio	Fecha_Alta	Nombre	DNI	Localidad
0	15000	2023-03-01	María González	28456123	Rosario
1	22000	2023-05-14	Carlos Alberto Rodríguez	31782456	Rosario
2	12000	2024-01-08	Roberto Sánchez	33841290	Rosario

El resultado final contiene sólo a los beneficiarios que pudieron vincularse con un registro del padrón con un `score` superior al umbral definido. En este ejemplo, quedan afuera quienes no alcanzan el umbral y también quienes no tienen contraparte en el padrón.

! Importante

El *fuzzy join* es una herramienta muy útil, pero no infalible. En aplicaciones reales, conviene revisar manualmente los casos con `scores` intermedios. Cuando se dispone de un identificador único confiable en ambas bases —por ejemplo, el DNI— siempre es preferible usarlo como clave de unión. El *fuzzy join* es un recurso cuando esa alternativa no está disponible (como en este ejemplo, donde por algún motivo no contamos con esa información en el dataset `df_beneficiarios`).

Sobre el preprocesamiento de textos

Cuando usamos `processor = utils.default_process`, las cadenas se transforman antes de ser comparadas. En particular, este preprocesamiento realiza:

- conversión a minúsculas
- eliminación de caracteres no alfanuméricos
- eliminación de espacios en blanco extra

Esto ayuda a evitar diferencias triviales como `'María gonzález'` vs. `'maría gonzález'` o problemas con signos de puntuación.

Sin embargo, este procesamiento NO elimina ni normaliza tildes (acentos).

Por ejemplo, las cadenas:

`'María González'`

`'Maria Gonzalez'`

siguen siendo distintas, porque los caracteres `í` e `i` (o `á` y `a`) no son equivalentes. Esto explica por qué, en estos casos, el score de similitud es menor a 100.

i Algo más sobre el tratamiento de tildes

Si se desea que las tildes no afecten la comparación, es necesario aplicar una normalización adicional, por ejemplo, eliminándolas con el módulo `unicodedata`.